

Architettura degli Elaboratori

Le istruzioni della CPU - Controllo del flusso



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits:

Alessandro Checco, Andrea Sterbini,

Iacopo Masi,

Claudio di Ciccio

[S&PdC] Capitolo 2



Argomenti

Argomenti della lezione

- Istruzioni Logiche
- Istruzioni per prendere decisioni (if then else, cicli)
- Il simulatore RARS
- Esercitazione

Scaricate il **simulatore RARS** da:

<https://github.com/rarsm/rars/releases/download/latest/rars-flatlaf.jar>

Per l'esame useremo la versione originale (vecchia)

https://github.com/TheThirdOne/rars/releases/download/v1.6/rars1_6.jar

- è scritto in Java (**gira su qualsiasi OS, su Windows serve jdk23**)
- è **estensibile** (diversi plug-in per esaminare il comportamento della CPU e della Memoria)
- è un IDE integrato:
 - **Editor con evidenziazione della sintassi** assembly ed **autocompletamento** delle istruzioni
 - **Help completo** (istruzioni, syscall ...)
 - **Simulatore** con possibilità di **esecuzione passo-passo** e uso di **breakpoint**
 - Permette l'**esecuzione da riga di comando** e la **compilazione di più file**

Istruzioni per prendere decisioni (istruzioni condizionali)



SAPIENZA
UNIVERSITÀ DI ROMA

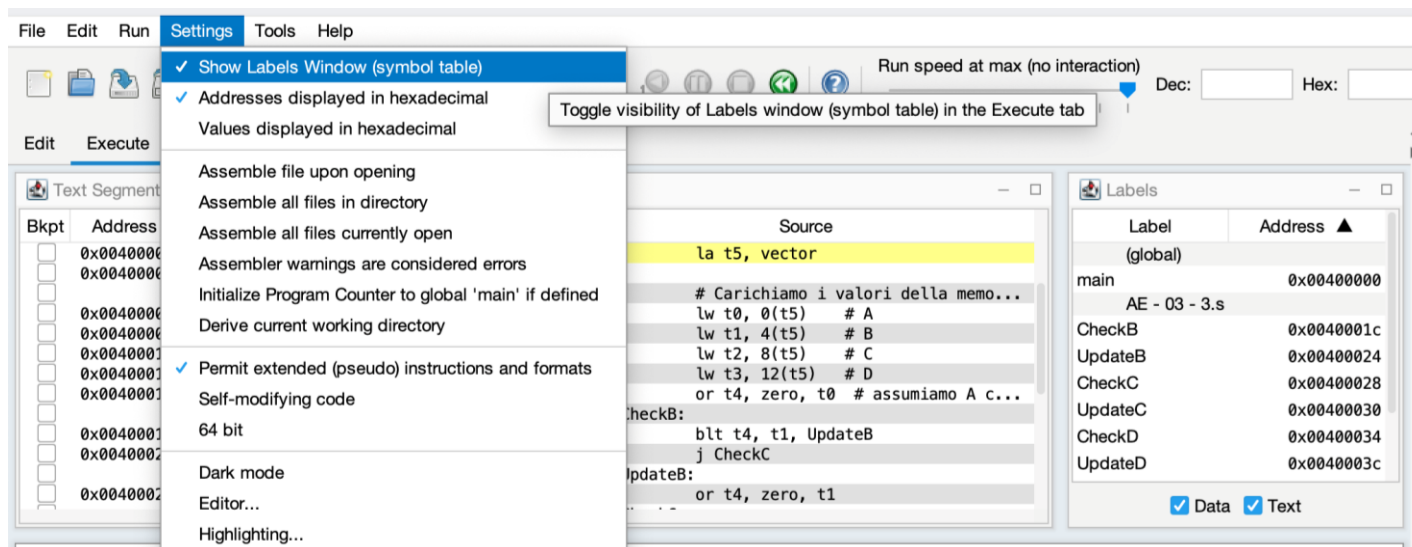
Comparazione e salto condizionato

- beq s1,s2,C
- bne s1,s2,C
- bge s1,x0,C
- bge x0,s1,C
- blt s1,x0,C
- bgt s1,x0,C

Branch on equal
Branch on not equal
Branch on less-than or eq. 0
Branch on greater-than or eq.0
Branch on less than 0
Branch on greater than 0

C'è una ETICHETTA
che identifica
un'istruzione

Pseudocodice delle azioni sottostanti
PC += 4
if (\$s1 == \$s2) { PC += C }



Comparazione e salto condizionato

- `beq s1,s2,C` # Branch on equal
- `bne s1,s2,C` # Branch on not equal
- `bge s1,x0,C` # Branch on less-than or eq. 0
- `bge x0,s1,C` # Branch on greater-than or eq.0
- `blt s1,x0,C` # Branch on less than 0
- `bgt s1,x0,C` # Branch on greater than 0

- `slt s0,s1,s2` # Set s0 to 1 if s1 l.than s2
- `slti s0,s1,10` # Set s0 to 1 if s1 l.than 10

RARS



SAPIENZA
UNIVERSITÀ DI ROMA

RARS - avvio

Windows

Se avete JVM, potete semplicemente cliccarci sopra (**doppio click**). Dovrebbe funzionare

Mac OS - Linux

Su Mac OS e Linux il doppio click funziona, ma in entrambi i casi (Mac e Linux) si può lanciare anche da riga di comando così:

```
(base) [~]$ java --jar /Applications/Rars.jar
```

Il simulatore RARS

HA:\shortcut-targets-by-id\1HXFX2Rjd195ZzNzTbtp6-g-3vbqXQXP\Materiale Comune\Slide comuni\AE - 03 - 3.s* - RARS 1.6

File Edit Run Settings Tools Help

Run speed at max (no interaction)

AE - 03 - 3.s*

```
1 # selezionare il massimo da un vettore
2 .globl main
3
4 .data
5     vector: .word 4, 3, -5, 500
6     rez: .word 0
7
8 .text
9
10 main:
11     # Carichiamo l'indirizzo di "vector"
12     la t5, vector
13     # Carichiamo i valori della memoria nei registri
14     l add, Addition
15     l addi, Addition immediate
16     lw t2, 0(t5) # C
17     lw t3, 12(t5) # D
18
19     or t4, zero, t0 # assumiamo A come massimo iniziale
20
21 CheckB:
22     blt t4, t1, UpdateB
23     jal CheckC
24 UpdateB:
25     or t4, zero, t1
26 CheckC:
27     blt t4, t2, UpdateC
28     jal CheckD
29 UpdateC:
30     or t4, zero, t2
31 CheckD:
32     blt t4, t3, UpdateD
33     jal End
34 UpdateD:
35     or t4, zero, t3
36 End:
```

EDITOR

Auto Completamento

Syntax highlight

REGS

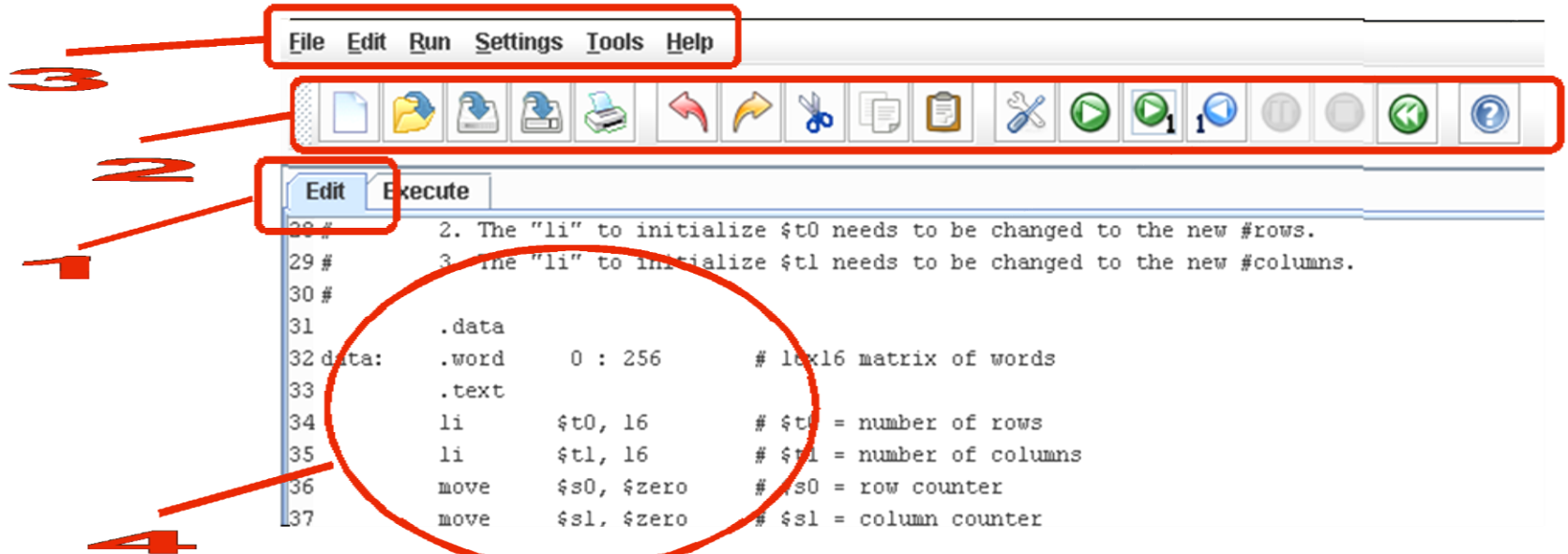
Registers	Floating Point	Control and Status
Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x7ffffffe
gp	3	0x10008000
tp	4	0x00000000
t0	5	0x00000000
t1	6	0x00000000
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x00000000
a1	11	0x00000000
a2	12	0x00000000
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x00000000
s2	18	0x00000000
s3	19	0x00000000
s4	20	0x00000000
s5	21	0x00000000
s6	22	0x00000000
s7	23	0x00000000
s8	24	0x00000000
s9	25	0x00000000
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000000
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000000
pc		0x00400000

Messages Run I/O

Clear

CONSOLE
Stampe / Input da tastiera

RARS: la toolbar



1 Interfaccia a tab (uno per ciascun file aperto, più il simulatore)



2 Toolbar (Compilazione, Esecuzione, Esecuzione passo-passo in avanti e indietro, Reset)

3 Menu

4 Finestra dell'editor

Assembly RISC-V

Direttive principali per l'assemblatore

.data	definizione dei dati statici
.text	definizione del programma
.ascii	stringa terminata da zero
.byte	sequenza di byte
.double	sequenza di double
.float	sequenza di float
.half	sequenza di half words
.word	sequenza di words
.globl <i>sym</i>	dichiara il simbolo come globale e può essere referenziato da altri file

Codici mnemonici delle istruzioni

add, sub, beq ...

Pseudoistruzioni: istruzioni non implementati in hardware, ma che possono essere tradotte in istruzioni macchina.

Es. mv x10, x11 → addi x10, x11, 0

Codifica mnemonica dei **registri**

a0, sp, ra ... f0, f31

Etichette (per calcolare gli indirizzi relativi)
nome:

L'assemblatore converte

-dal testo del programma in assembly

-il programma in codice macchina

-Dalle etichette calcola gli indirizzi

Dei salti relativi

Delle strutture dati in memoria (offset)

Dei salti assoluti

NOTA: le strutture di controllo del flusso del programma vanno realizzate «a mano» usando i **salti condizionati e le etichette**

Istruzioni condizionali

Alla fine di questa lezione sapremo come il calcolatore implementa i costrutti maggiormente usati in una programmazione di alto livello come C o C++:

Mappiamo in assembly le seguenti istruzioni:

1. If then else
2. Iterazioni (cicli)
 1. Do while loop
 2. While loop
 3. For loop
3. If then else con «casi multiple» (switch statement)

Prendere decisioni (if then else)

Esempio C

```
if ( X >= 0 ) {  
    // codice da eseguire se il test è vero  
  
} else {  
    // codice da eseguire se il test è falso  
  
}  
  
// codice seguente
```

Esempio Assembly

```
.text  
# uso il registro t0 per la var. X  
bltz t0, else      # test X < 0  
  
# codice da eseguire se il test è vero  
  
j endif            # esco dall'IF  
  
# codice da eseguire se il test è falso  
  
endif:  
# codice seguente
```

NOTA: il test inserito è l'opposto dell'originale

Prendere decisioni (if then else)

Esempio C

```
if ( X >= 0 ) {  
    // codice da eseguire se il test è vero  
  
} else {  
    // codice da eseguire se il test è falso  
  
}  
  
// codice seguente
```

Esempio Assembly

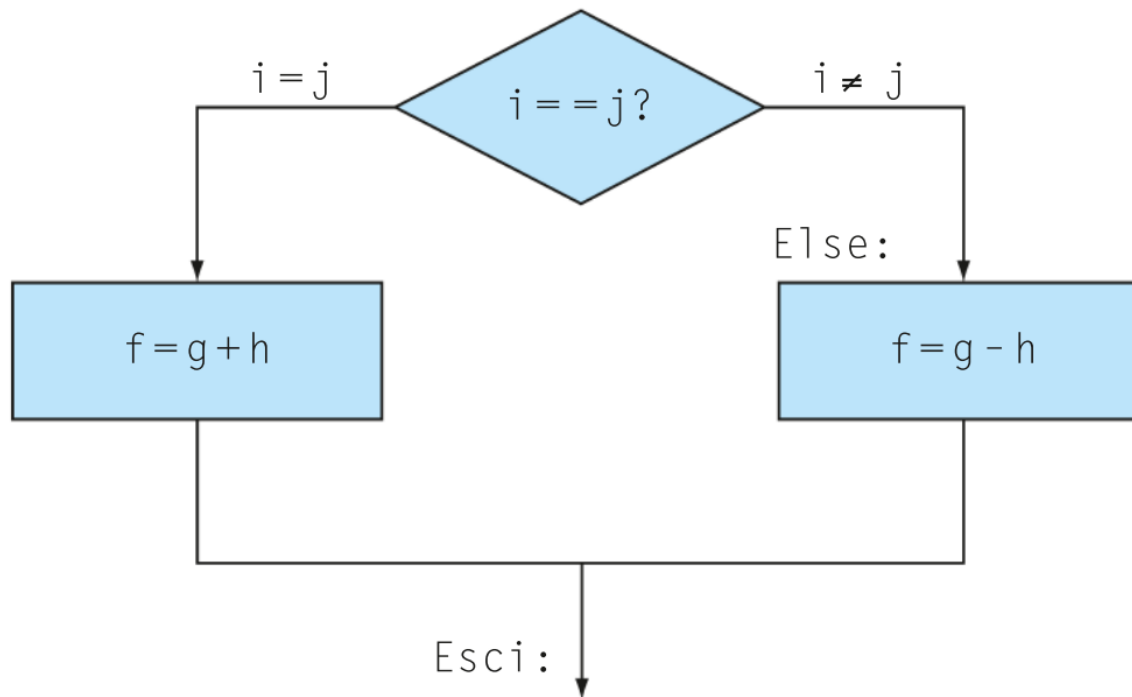
```
.text  
# uso il registro t0 per la var. X  
bltz t0, else      # test X <= 0  
  
# codice da eseguire se il test è vero  
  
j endIf            # esco dall'IF  
  
# codice da eseguire se il test è falso  
  
endif:  
# codice seguente
```



NOTA: il test inserito è l'opposto dell'originale

Ancora, Prendere decisioni (if then else)

```
if (i == j) f = g + h; else f = g - h;
```



Prendere decisioni (if then else)

```
if (i == j) f = g + h; else f = g - h;
```

```
bne s3,s4,Else      # vai a Else se i ≠ j
```

Prendere decisioni (if then else)

```
if (i == j) f = g + h; else f = g - h;
```

```
bne s3,s4,Else      # vai a Else se i ≠ j
```

```
add s0,s1,s2        # f = g + h (saltata se i ≠ j)
```


Prendere decisioni (if then else)

```
if (i == j) f = g + h; else f = g - h;
```

```
bne s3,s4,Else      # vai a Else se i ≠ j  
add s0,s1,s2        # f = g + h (saltata se i ≠ j)  
j Esci              # vai a Esci
```

Prendere decisioni (if then else)

```
if (i == j) f = g + h; else f = g - h;
```

```
bne s3,s4,Else      # vai a Else se i ≠ j  
add s0,s1,s2        # f = g + h (saltata se i ≠ j)  
j Esci              # vai a Esci  
Else: sub s0,s1,s2.  # f = g - h (saltata se i = j)  
Esci:
```

Realizzare Iterazioni (ciclo do while)

Esempio C

```
do {  
  
    // codice da ripetere se x != 0  
    // il corpo del ciclo DEVE aggiornare x  
  
} while (x != 0);  
  
// codice seguente
```

Esempio Assembly

```
.text  
    # uso il registro t0 per l'indice x  
  
do:  
  
    # codice da ripetere  
  
    bnez t0, do    # test x != 0  
  
    # codice seguente
```

NOTA: il test inserito è uguale all'originale

Realizzare Iterazioni (while do)

Esempio C

```
while (x != 0) {  
  
    // codice da ripetere se x != 0  
    // il corpo del ciclo DEVE aggiornare x  
}  
  
// codice seguente
```

Esempio Assembly

```
.text  
    # uso il registro t0 per l'indice x  
while:  
    beqz t0, endWhile    # test x == 0  
  
    # codice da ripetere  
  
    j while                # loop  
endWhile:  
    # codice seguente
```

NOTA: il test inserito è l'opposto dell'originale

Realizzare Iterazioni (for loop)

Esempio C

```
for (i=0 ; i<N ; i++)  
{  
    // codice da ripetere  
}  
  
// codice seguente
```

Esempio Assembly

```
.text  
  
# uso il registro t0 per l'indice i  
# uso il registro t1 per il limite N  
xor t0, t0, t0          # azzero i  
li t1, N                # limite del ciclo  
  
cicloFor:  
bge t0, t1, endFor    # test i >= N  
# codice da ripetere  
addi t0, t0, 1          # incremento di i  
j cicloFor             # loop  
  
endFor:  
# codice seguente
```

NOTA: il test inserito è l'opposto dell'originale

SWITCH CASE

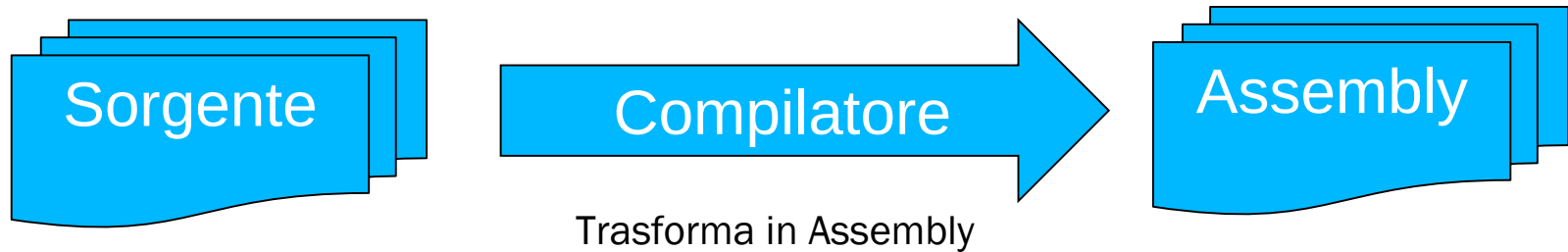
Esempio C

```
switch (A) {  
case 0:    // codice del caso 0  
    break;  
case 1:    // codice del caso 1  
    break;  
// altri casi  
case N:    // codice del caso 3  
    break;  
}  
// codice seguente
```

Esempio Assembly

```
.text  
    lbu t0, A           # selezioniamo caso A  
    slli t0, t0, 2       # A*4  
    la t1, dest         # indirizzo primo case  
    add t1, t0, t1       # indirizzo selezionato  
    lw t1, (t1)          # carico indirizzo  
    jr t1               # salto a registro  
  
    caso0:              # codice del caso 0  
        j endSwitch  
    caso1:              # codice del caso 1  
        j endSwitch  
    casoN:              # codice del caso N  
        j endSwitch  
    endSwitch:          # codice seguente  
  
.data  
dest: .word caso0, caso1, ..., casoN  
A:    .byte
```

Compilatore / Assemblatore



Istruzioni/espressioni di alto livello => gruppi di istruzioni ASM

variabili temporanee => registri

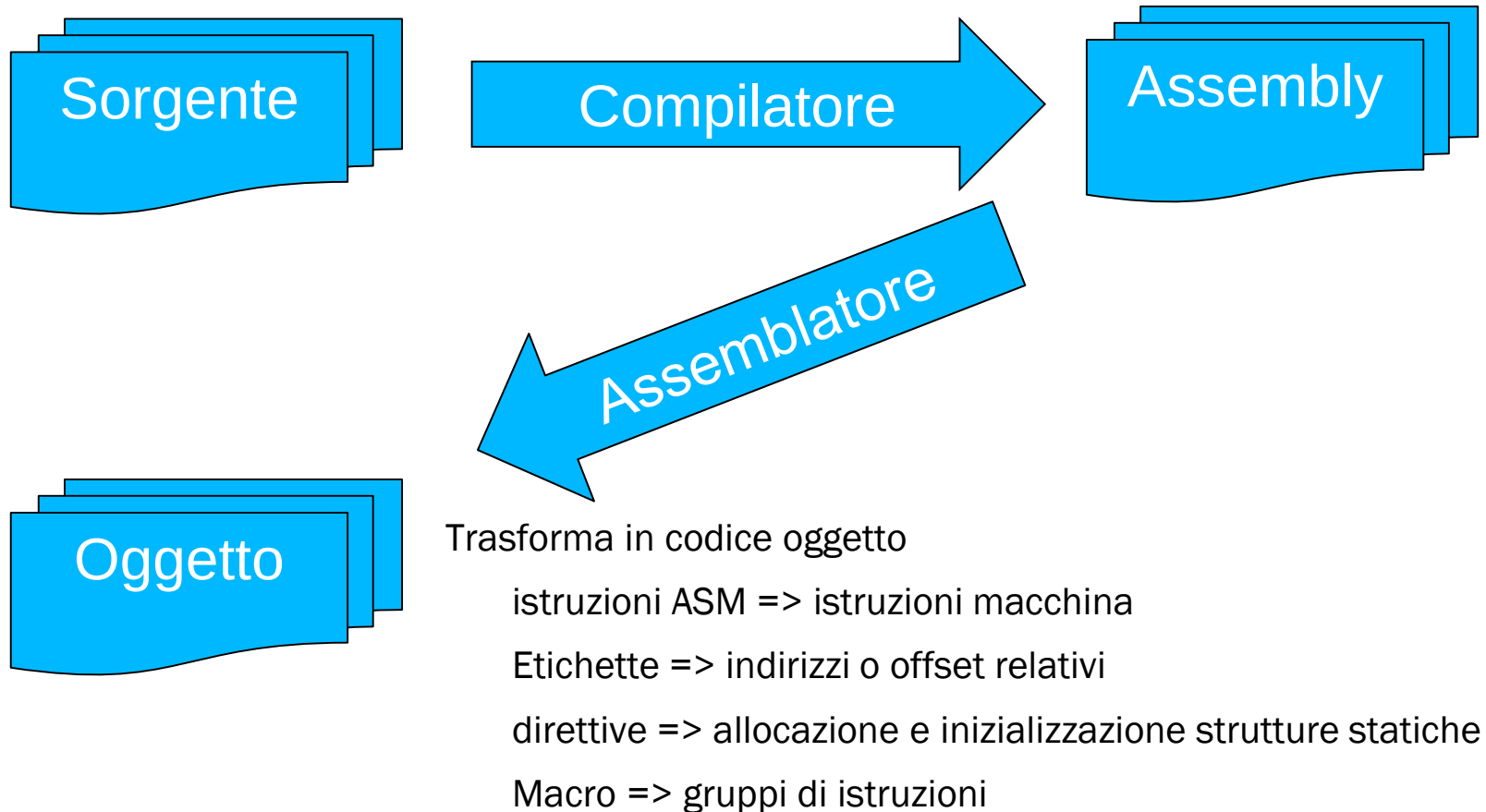
Variabili globali e locali => etichette e direttive

Strutture di controllo => salti ed etichette

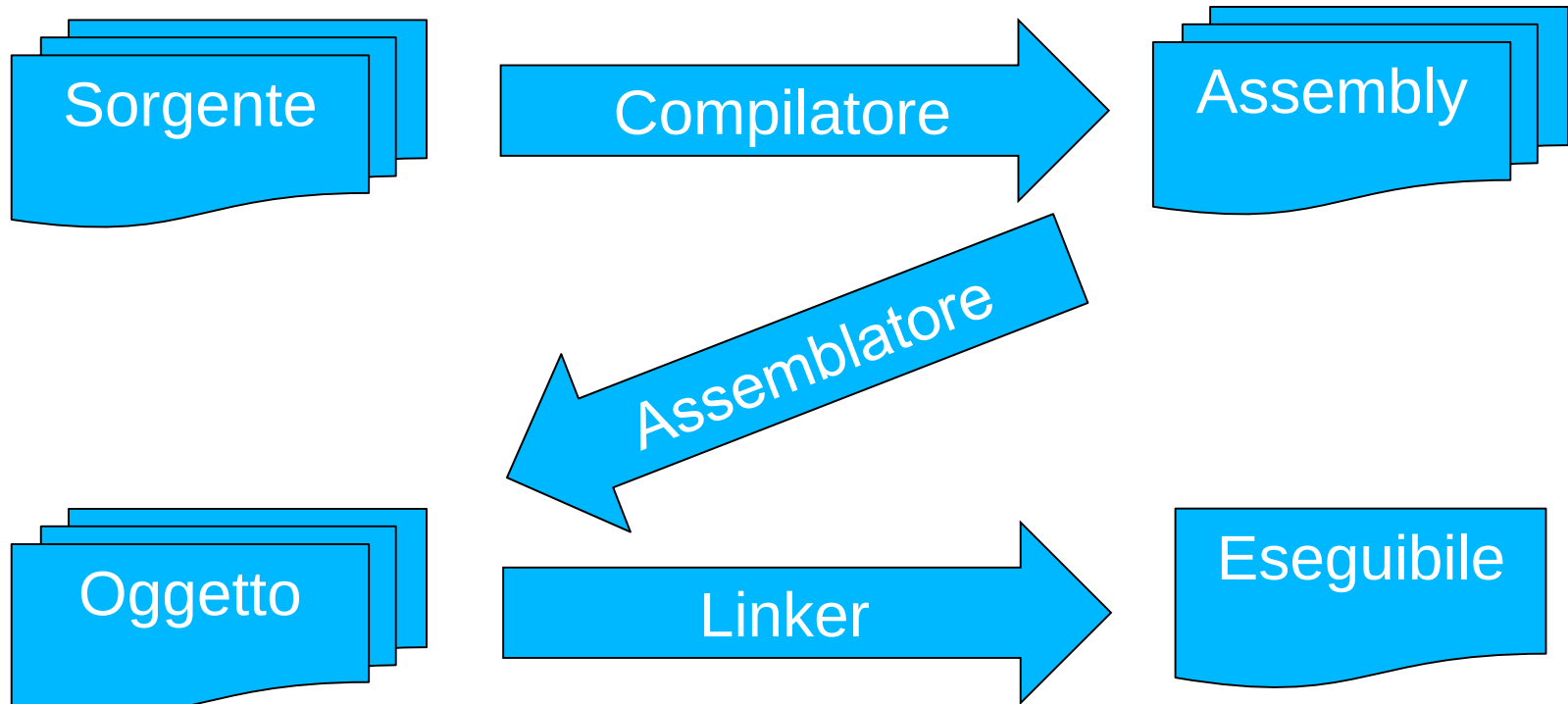
funzioni e chiamate => etichette e salti a funzione

chiamate a funzioni esterne => tabella x linker

Compilatore / Assemblatore



Compilatore / Assemblatore



Definisce la pos. In memoria delle strutture dati statiche e del codice

«Collega» i riferimenti a

chiamate di funzioni esterne => salti non relativi

strutture dati esterne => indirizzamenti non relativi

La direttiva **globl**

La useremo ma non è essenziale quando si lavora con un solo file come nel nostro caso.

E' utile quando abbiamo **più file da gestire** e abbiamo delle parti del codice che referenziano etichette che sono in file diversi.

Chi dice all'assemblatore dove andare a prendere le etichette non definite in quel file?
Possiamo farlo con la direttiva **globl**.

```
.globl main
.text
main:
    addi t1,zero,-100
```

File main.S

```
....
.text

j main

.....
.....
```

File file_B.S

```
% as -o main.o main.S
```

 Assembla e crea file oggetto

```
% as -o file_B.o file_B.s
```

 Assembla e crea file oggetto

```
% ls -o main.bin file_B.o main.o
```

 Linka il tutto e crea **eseguibile** (su windows sarebbero i file .exe)

La direttiva `globl`

La useremo ma non è essenziale quando si lavora con un solo file come nel nostro caso.

E' utile quando abbiamo **più file da gestire** e abbiamo delle parti del codice che referenziano etichette che sono in file diversi.

Chi dice all'assemblatore dove andare a prendere le etichette non definite in quel file?
Possiamo farlo con la direttiva `globl`.

```
.globl main
.text
main:
    addi t1,zero,-100
```

File main.S

```
....
.text

j main

.....
.....
```

File file_B.S

Se i global non sono specificati bene
si verifica un errore al momento di
Linking (quando si crea eseguibile)
Perché il linker non sa come risolvere
label_fileB

```
% as -o main.o main.S
```

 Assembla e crea file oggetto

```
% as -o file_B.o file_B.s
```

 Assembla e crea file oggetto

```
% ls -o main.bin file_B.o main.o
```

 Linka il tutto e crea **eseguibile** (su windows sarebbero i file .exe)

Le pseudoistruzioni

<code>auipc x6,64528</code>	4:	la t1, dest
<code>addi x6,x6,-8</code>		

- La mia istruzione sulla destra è stata mappata in due istruzioni sulla sinistra dall'assemblatore
- `auipc` somma in t1 PC + i 20 bit più significativi della distanza tra PC e dest
- somma in t1 i 12 bit meno significativi della distanza tra PC e dest

Le pseudoistruzioni

```
bge x0,x6,4
```

```
8:
```

```
blez t1,caso0
```

- Il **blez** (branch if less than or equal to zero) è fittizio. E' in realtà stato implementato con **bge**.
- Abbiamo già incontrato diverse istruzioni reali dove il registro zero è già comparso diverse volte. **Velocizzare le situazioni più comuni** quindi è importante che la codifica della zero sia sempre disponibile

Es.: trova il max di un vettore

Esempio C

```
// definizione dei dati
int vettore[6] = { 11, 35, 2, 17, 29, 95 };
int N = 6;

int max = vettore[0];
// scansiono il vettore
for (i=1 ; i<N ; i++) {

    int el = vettore[i];    // el. corrente
    if (elemento > max)
        max = elemento;

}
```

Esempio Assembly

```
.data
vettore: .word 11, 35, 2, 17, 29, 95
N:      .word 6
.text
    la    s0, vettore
    lw    t0, 0(s0)        # max → $t0
    lw    t1, N           # N → $t1
    li    t2, 1            # i = 1
for: bge t2, t1, endFor
    slli   t3, t2, 2        # i*4
    add    t3, t3, s0       # t3 = vettore+i*4
    lw     t4, (t3)         # el. = vettore[i]
    ble    t4, t0, else    # if (el >= max)
    mv     t0, t4           # max = el.
else:
    addi   t2, t2, 1        # i++
    j for
endFor:
```

Compito per casa



1. Installare **RARS** e prendere familiarità con interfaccia
2. Scrivere i programmi visti a lezione e provare ad eseguirli e debuggarli passo passo
 - Controllare come cambiano i registri sulla destra in base ai passi svolti.
 - Ispezionare come cambiano le pseudo-istruzioni immesse nelle istruzioni che poi svolge veramente il calcolatore
3. Partendo dal programma che trova il max in un vettore scrivere un programma in linguaggio assembly RISC-V con RARS che dato un vettore ingresso **vector** e la sua dimensione **N** calcoli due somme dei numeri del vettore.
 1. La prima somma deve sommare i valori del vettore di indice **dispari**. (Indice parte da 1)
 2. La seconda somma deve sommare i valori di un vettore con **indice pari**. (Indice parte da 0)



Registro	Nome
x0	zero
x1	ra
x2	sp
x3	gp
x4	tp
x5-x7	t0-t2
x8	s0 / fp
x9	s1
x10-x11	a0-a1
x12-x17	a2-a7
x18-x27	s2-s11
x28-x31	t3-t6

Vector .word **[0]** **[1]** **[2]** **[3]** **[4]** **[5]** **[6]**
 4 -1 5 500 0 10000 -256

N .word 5

Somme: .word 0, 0

Una volta scritto con questi dati cambiate **N** e controllate se continua a funzionare